# Deciding subtype entailment for bottom and binary product over arbitrary trees

Author and affiliation placeholders

Research draft, 20 September 2026

**Research manuscript — draft for review.** Authors and affiliations: [**placeholders**]. Revised: 2026-09-20. Formal source checkpoint: c946d86 (claim C081). Licensing: pending the owner's decision. This draft makes no publication-priority claim.

## Abstract

We give a terminating decision procedure for entailment between finite type expressions built from variables, bottom, and one covariant binary product. Variables range over arbitrary trees, including infinite nonregular trees. The proof reduces failure of entailment to a finite automaton problem with constraints on agreeing suffixes. The automata obtained from subtyping have an absorption property that separates this problem into a terminal case and an infinite-continuation case. The latter admits a finite periodic monitor. For the terminal case, we reverse a candidate word and use its border rank. Although reference words can grow without a known useful length bound, their first rank milestones form a bounded sequence of first returns. A first return adds a directed strip with one entrance to a coloured prefix graph. We give a finite construction that pulls a whole existential regular-path query back through this extension. Products of observers over each physical interval preserve the single shared generated word. Iterating this construction a bounded number of times and evaluating the resulting query on two-vertex graphs decides the terminal case. The construction, both semantic directions, and its composition with type normalization are checked in Lean. No numerical witness bound is assumed. Practical efficiency, stronger quantitative bounds, and extensions of the type signature are separate questions.

# Contents

**Abstract** 1

**1. The theorem and its semantic boundary** 3
A small example . . . 3

**2. Normalize expressions without changing their models** 4

**3. Canonical propagation and finite stack summaries** 4
3.1 The least model generated by seeds . . . 4
3.2 Effective pushdown reachability . . . 5
3.3 One finite flagged automaton for all separating words . . . 6

**4. Absorption separates two complete cases** 7

**5. Reversal, border rank, and first-return milestones** 7

**6. Prefix graphs and the exact first-return extension** 9
6.1 A finite coloured graph for a concrete reference . . . 9
6.2 Generate one first return . . . 9
Worked graph example: an overlapping return . . . 10

**7. Positive directed queries and a finite preimage construction** 10
7.1 The query language . . . 10
7.2 Why finitely many diagrams cover every satisfying extension . . . 11
7.3 Pull back the old core . . . 12
7.4 Encode marks and the portal edges . . . 13
7.5 One common generator on every physical interval . . . 13
7.6 Assemble the finite query . . . 14
7.7 Proof of coverage: a promoted model gives a base model . . . 14
7.8 Proof of reconstruction: a base model gives one promoted model . . . 15
A complete two-observer diagram . . . 15
Why sharing is indispensable . . . 16
Why direction is indispensable . . . 16

**8. Finish without another return** 16

**9. Decide terminal witnesses by finite semantic promotion** 17

**10. Decide the infinite-continuation branch** 18
10.1 The periodic suffix criterion . . . 18
10.2 A complete finite reverse monitor . . . 18
10.3 Return from an infinite stream to a finite witness . . . 19

**11. Compose the complete decision** 19
A computable bound without a useful numerical estimate . . . 20

**12. Validation, provenance, and present limits** 20

**13. Related work and attribution status** 21

# 1. The theorem and its semantic boundary

Let $V$ be a finite set of variables and let expressions be generated by

$$e ::= x \in V \mid \bot \mid e \times e.$$

Write $\Sigma = \{0, 1\}$ and $\epsilon$ for the empty word. A type is a prefix-closed set $X \subseteq \Sigma^*$ of **product-labelled addresses**. The empty set represents bottom. A product is

$$X \times Y = \{\epsilon\} \cup \{0u : u \in X\} \cup \{1u : u \in Y\}.$$

Subtyping is inclusion. A valuation $\rho : V \to \mathcal{T}$ extends to expressions by these operations. Given a finite list $\Gamma$ of inequalities,

$$\Gamma \vDash s \leq t \quad \iff \quad \forall \rho \, [\, (\forall (u \leq v) \in \Gamma, [\![u]\!]_\rho \subseteq [\![v]\!]_\rho) \Rightarrow [\![s]\!]_\rho \subseteq [\![t]\!]_\rho \,].$$

There is no computability, regularity, finite-depth, or decidable-membership restriction on $\rho$. The query $s \leq t$ is not inserted into $\Gamma$. Inconsistent constraints entail every query.

**Theorem P1 (C081).** There is a total executable Boolean procedure $D$ such that, for every finite $\Gamma, s, t$,

$$D(\Gamma, s, t) = \mathsf{true} \quad \iff \quad \Gamma \vDash s \leq t.$$

The Lean input is a finite list over `Expr (Fin n)`. Indexing a finite named variable set gives this input effectively; repeated premises do not change its interpretation. The theorem does not certify an arbitrary text parser.

The address-set representation includes the ordinary labelled-tree boundary conditions. The root exists even for bottom; a nonroot address $ua$ exists exactly when $u$ is a product address. All defined nonproduct nodes are bottom. Conversely, a rooted bottom/product diagram in which exactly product nodes have two children determines a prefix-closed product set. These two descriptions agree. The full set $\Sigma^*$ is a greatest type in this domain, but it is an infinite tree of products, not an explicit top-labelled leaf. Adding such a leaf changes the signature and is not covered by P1.

### A small example

For $\Gamma = \{x \leq y\}$, we have $\Gamma \vDash x \times \bot \leq y \times \bot$ by covariance. The reverse query fails: assign $x = \bot$ and $y = \bot \times \bot$. The left product in the reverse query has product addresses $\{\epsilon, 0\}$, while the right product has only $\{\epsilon\}$. Address 0 separates them.

Infinite models are essential to the stated domain. Let $Z = \{0^j : j \geq 0\}$. Then $Z = Z \times \bot$ in address-set notation. Thus $\{x \times \bot \leq x\}$ is satisfiable, and does not entail $x \leq \bot$. It has no finite-tree model: its premise forces the root of $x$, then its left child, then every address $0^j$. Finite-tree entailment would make every query true for this input. This is a concrete reason not to replace arbitrary trees by finite trees in P1.

## 2. Normalize expressions without changing their models

Give each compound expression a fresh name, include the compound expressions in the query as well as those in the premises, and use a name $b$ for bottom when it occurs. For a product name $z$ representing $u \times v$, impose both

$$z \leq u \times v, \qquad u \times v \leq z.$$

Impose $b \leq \bot$. Original variables retain their identities; named premises become inequalities between names. The checked allocator may retain duplicate product slots. Their defining constraints give each the same interpretation; sharing is not required for correctness. The four resulting clause forms are

$$x \leq y, \qquad x \leq y \times z, \qquad y \times z \leq x, \qquad x \leq \bot.$$

**Lemma P2 (exact naming).** Every original model extends to a model of the normalized constraints by assigning each new name the value of its expression. Conversely, restricting any normalized model to the original variables gives an original model, and every expression name has its expression's value. The original and normalized entailment queries are therefore equivalent.

*Proof.* The extension satisfies each pair of defining inequalities by equality of interpretations. In the other direction, $b \subseteq \emptyset$ forces $b = \emptyset$. The two product inequalities force the product name to equal the product of the two component names, by antisymmetry of set inclusion. Induction over finite expression syntax gives the interpretation of every name. Apply this equality to each premise and to both query expressions. Nothing in this induction unfolds a possibly infinite value. It is an induction on input syntax only. □

We henceforth fix a normalized constraint list $C$ on $n$ controls and a variable query $\alpha \leq \beta$.

## 3. Canonical propagation and finite stack summaries

### 3.1 The least model generated by seeds

A fact $(x, u)$ means that $u$ is a product address of variable $x$. The constraints induce these rules, for every word $u$:

| Clause | Propagation |
|---|---|
| $x \leq y$ | $(x, u) \to (y, u)$ |
| $x \leq y \times z$ | $(x, 0u) \to (y, u)$ and $(x, 1u) \to (z, u)$ |
| $y \times z \leq x$ | $(y, u) \to (x, 0u)$ and $(z, u) \to (x, 1u)$; also force $(x, \epsilon)$ |
| $x \leq \bot$ | forbid every fact $(x, u)$ |

Let $F_0$ be the controls whose root is forced by a lower-product clause, and let $N_0$ be the denied controls. For a seed family $S$, let $\mathrm{cl}(S)$ be its closure under the five directed propagation rules in the table. Use all forced roots as seeds. To ask whether an address $w$ separates $\alpha$ from $\beta$, also seed $(\alpha, u)$ for **every prefix** $u$ of $w$. Call this finite seed set $S_{\alpha,w}$.

**Lemma P3 (canonical separation).** The query fails exactly when some finite word $w$ satisfies

$$\mathrm{cl}(S_{\alpha,w}) \cap (N_0 \times \Sigma^*) = \emptyset, \qquad (\beta, w) \notin \mathrm{cl}(S_{\alpha,w}). \tag{3.1}$$

*Proof.* Each propagation rule preserves truth in every model containing its seeds. Induction on a derivation proves that the closure is contained in every such model. The closure is prefix-closed: truncate a derivation at an arbitrary address length. Epsilon and pop steps preserve the corresponding truncated step; a push truncated to length zero is justified by the forced root of its target. All other truncated pushes use the induction hypothesis. Thus the closure defines trees. If no denial is reached, these trees satisfy all four clause forms, directly by the rules, including the forced root in a lower clause. They contain $w$ at $\alpha$ and omit it at $\beta$ exactly as in (3.1). Conversely, any failure of set inclusion has a finite separating address $w$. Prefix closure puts all its prefix seeds in the countermodel. Leastness then proves both conditions of (3.1). No finite-model assumption enters. □

### 3.2 Effective pushdown reachability

Ignore the forced roots and denials temporarily, keeping the epsilon, push, and pop rules as a finite system $W$ on configurations $(x, u)$, with the stack's top on the **left**. Reverse every rule to obtain $W^{\mathrm{op}}$; push and pop are interchanged, and epsilon edges reverse direction.

Let $E_W(p, q)$ mean a balanced excursion: the stack ends unchanged and is never read below its original contents. It is the least relation containing identities and epsilon edges and closed under composition and

$$p \ \mathrm{push}(a)\ r, \quad E_W(r, s), \quad s \ \mathrm{pop}(a)\ q \quad \Longrightarrow \quad E_W(p, q).$$

This is an effective relation on $n^2$ pairs. One may saturate these finite Horn rules. The current checked implementation instead tests membership in **every closed table**, by enumerating all subsets of the $n^2$ pairs. This very expensive choice is still finite and exact: the least closed relation is their intersection. The paper does not attribute a saturation running time to that implementation.

Define $\mathrm{Pop}_W(p, u, q)$ recursively: for $u = \epsilon$, use $E_W(p, q)$; for $u = av$, take a balanced excursion, pop $a$, then use Pop on $v$. It means that the prefix $u$ can be removed, leaving any unchanged stack suffix. It is decidable from the finite balanced table.

**Lemma P4 (minimum-stack factorization).** A configuration $(p, u)$ reaches $(q, v)$ if and only if there are words $a, b, z$ and a control $r$ such that

$$u = az, \quad v = bz, \quad \mathrm{Pop}_W(p, a, r), \quad \mathrm{Pop}_{W^{\mathrm{op}}}(q, b, r). \tag{3.2}$$

*Proof.* In a finite run, choose a configuration at minimum stack height. Its stack $z$ is an unchanged suffix of both endpoint stacks. Before that configuration, each first descent past one stack letter is a pop, preceded by a balanced excursion. This gives the first Pop derivation. Reverse the remaining run to obtain the second. Conversely, a Pop derivation is valid above any suffix $z$. Follow the first and then the reverse of the second. The empty run, empty popped prefixes and intervening balanced excursions are included. This argument also proves the finite recursive Pop test correct. □

Existence of *some* word $u$ with $\mathrm{Pop}_W(p, u, q)$ is finite control reachability using balanced excursions and arbitrary labelled pops. Hence we can decide whether a finite seed set reaches any denied

control at any stack height. Applying this to the forced roots decides satisfiability of $C$. This test exhausts finite relations, not a bounded portion of an infinite stack.

### 3.3 One finite flagged automaton for all separating words

For a prefix $u$ define

$$B(u) = \{r : \mathrm{Pop}_W(\alpha, u, r)\}, \qquad A(u) = \{r : \mathrm{Pop}_{W^{\mathrm{op}}}(\beta, u, r)\}.$$

Use all pairs $(B, A)$ of control subsets as physical states: there are $4^n$. The initial state is $(B(\epsilon), A(\epsilon))$. On a bit $a$, replace either subset $S$ by

$$\mathrm{next}_W(S, a) = \{q : \exists p \in S, r, p \ \mathrm{pop}(a)\ r \ \wedge E_W(r, q)\}, \tag{3.3}$$

using $W^{\mathrm{op}}$ for the second component. The last-letter decomposition of Pop proves that the reachable state after $u$ is exactly $(B(u), A(u))$. This interpretation is asserted for reachable states; arbitrary enumerated subsets need not already be balanced-closed.

Define the fixed danger set

$$D_0 = \{r : \exists d \in N_0, v, \mathrm{Pop}_{W^{\mathrm{op}}}(d, v, r)\}.$$

Its membership is decidable by the finite control test above. The automaton's annotations are:

- $(B, A)$ is final iff $A \cap F_0 = \emptyset$;
- it is Bad iff $C$ is unsatisfiable or $B \cap D_0 \neq \emptyset$;
- the ordered pair $((B, A), (B', A'))$ is flagged iff $B \cap A' \neq \emptyset$.

For any finite partial deterministic binary automaton $I = (Q, \delta, s, F, \mathrm{Bad}, G)$, call $w$ a **witness** when its whole run $c_0, \ldots, c_N$ exists, avoids Bad, ends in $F$, and

$$\forall\, 0 \le i \le j \le N : \quad w[j:] \text{ is a prefix of } w[i:] \implies (c_i, c_j) \notin G. \tag{3.4}$$

The diagonal $i = j$, the empty suffix $j = N$, and the initial cut $i = 0$ are all part of this definition. In particular, a diagonally flagged state is unsafe.

**Lemma P5 (exact summary bridge).** In the constructed instance, $w$ is a witness iff it satisfies (3.1). Consequently, $C \not\vdash \alpha \le \beta$ iff the constructed instance has a witness.

*Proof.* A forced root reaches $(\beta, w)$ exactly when $A(w)$ meets $F_0$, by reversing its run. For a prefix seed $(\alpha, u)$, apply (3.2) to a run ending at $(\beta, w)$. Write the popped-prefix lengths as $i, j$. Since $u$ is a prefix of $w$ and their remaining suffixes agree, $i \le j$ and $w[j:]$ is a prefix of $w[i:]$. The common minimum-stack control lies in $B(w[:i]) \cap A(w[:j])$. Conversely, from these data choose the seed prefix $w[:i]w[j:]$ and apply (3.2); the agreement makes it a prefix of $w$. This proves the flagged-pair condition, including its boundaries. A prefix seed reaches some denied control iff a popped prefix reaches a member of $D_0$. Thus safety of every prefix is exactly the Bad check, together with the forced-root satisfiability check. These are all canonical sources. Apply P3. □

## 4. Absorption separates two complete cases

Say that an instance is **absorbing** when every defined successor $q$ of any final state is final and has an empty incoming flag column:

$$\delta(f,a)=q,\ f\in F \quad\Longrightarrow\quad q\in F\ \wedge\ \forall p\in Q,(p,q)\notin G. \tag{4.1}$$

**Lemma P6 (summary absorption).** The instance of Section 3 is absorbing, on all its physical states.

*Proof.* Every source of a pop in $W^{\mathrm{op}}$ is a target of a push in $W$, and hence belongs to $F_0$. A final state's $A$ avoids $F_0$, so (3.3) makes the next $A$ empty. Such a successor remains final; its incoming flags are empty because every intersection with its $A$ is empty. No reachable-state interpretation of an arbitrary subset is needed here. □

A state is *safe* if it is neither Bad nor diagonally flagged. Delete edges incident with unsafe states. In this safe graph, let $I_{\mathrm{term}}$ keep the original state set, initial state, Bad set and flags, and make final exactly the safe **original** finals that have no remaining outgoing edge. These finals are terminal. A new sink that was not originally final stays rejecting.

**Lemma P7 (terminal/cyclic split).** For an absorbing instance, a witness exists iff either $I_{\mathrm{term}}$ has a witness, or an original witness has a safe continuation reaching a nonempty safe cycle.

*Proof.* Appending a safe path to a witness preserves witnesshood. For a flagged comparison whose later cut lies in the old word, an agreement after extension would imply its forbidden old agreement by truncation. Every newer target has an empty incoming flag column by (4.1). Finality and safety also persist. Starting at a witness endpoint, grow a safe path. If it stops, it stops at a safe original final sink. If it reaches length $|Q|$, repeated cut states supply a safe stem and a nonempty cycle. In the first case the extended word is a witness of $I_{\mathrm{term}}$; in the second it has the stated cyclic continuation. Conversely, every terminalized witness lifts to the original instance, and the cyclic case already contains an original witness. The two cases need not be disjoint. □

Sections 5–9 decide terminal instances. Section 10 decides the cyclic case.

## 5. Reversal, border rank, and first-return milestones

Fix an instance $I$ with $k$ states and terminal finals. Reverse a nonempty witness $w$ to $R$, and put $r_j=c_{|w|-j}$. More generally a **safe labelled reference** consists of a nonempty word $B=b_0\cdots b_{m-1}$ and labels $r_0,\dots,r_m$ with

$$r_0\in F,\qquad \delta(r_{j+1},b_j)=r_j,$$

all labels outside Bad, and the reversed form of all comparisons in (3.4). Its last label need not be the original initial state $s$.

A border of a word is a prefix equal to a suffix; include the empty and full borders. Define the incoming column $\mathrm{col}(q)=\{p:(p,q)\in G\}$ and, at reference position $j$,

$$M_j=\bigcup_{b\text{ a border length of }B[:j]}\mathrm{col}(r_b). \tag{5.1}$$

Safety is exactly $r_j \notin M_j$ at every position, as well as avoidance of Bad. The full border checks the diagonal; the empty border checks flags into $r_0$. Indeed the comparison at original cuts $i \le j$ becomes the border of length $|w| - j$ of the reversed prefix of length $|w| - i$, with flag $(r_{|w|-i}, r_{|w|-j})$. Positive prefixes of a safe reference are safe. A safe reference ending at $s$ reverses to an original witness.

For $j > 0$, let $\pi(j)$ be the longest proper border length of $B[: j]$, and put $\pi(0) = 0$. Every proper border is a border of this longest proper border:

$$M_j = \mathrm{col}(r_j) \cup M_{\pi(j)}. \tag{5.2}$$

Let $d(j)$ be the number of positive borders of $B[: j]$, including the whole prefix. Thus $d(0) = 0$ and $d(j) = 1 + d(\pi(j))$ for $j > 0$.

**Lemma P8 (terminal rank bound).** Every prefix of a reversed terminal accepting run has $d(j) \le k - 1$.

*Proof.* Accepted words from a fixed state form a prefix-free language: a proper accepted prefix would leave the run at a terminal final with more letters to read. Fix an accepted suffix $x$ of length $\ell$. Map each border length $b$, including zero and $\ell$, to the state at cut $\ell - b$ in its accepting run. From this state the accepted remaining word is $x[\ell - b :] = x[: b]$. If two distinct lengths $b < c$ had the same state, that state would accept the two prefix-comparable words $x[: b]$ and $x[: c]$, contradicting prefix-freeness. There are therefore at most $k$ border lengths. The zero border maps to the terminal final and contributes one of them. Reversal preserves border lengths; a prefix of the reversed run is the reversal of such an accepted suffix. Removing the zero border leaves at most $k - 1$ positive borders. □

This bounds a nesting rank, not word length, the number of source visits, or the number of runs of equal symbols.

For a word $R$, apply $d$ and $\pi$ to its prefixes: $\pi(j)$ is the parent length of $R[: j]$. Let $m_h \le |R|$ be the first position with $d(m_h) \ge h$, when it exists. The first such position has rank exactly $h$, since its proper-border parent is an earlier position and $d = 1 + d \circ \pi$. In particular $m_1 = 1$.

**Lemma P9 (milestones are first returns).** If $m_h, m_{h+1}$ exist and $h > 0$, then $\pi(m_{h+1}) = m_h$, and the occurrence of $R[: m_h]$ ending at $m_{h+1}$ is its first occurrence ending strictly after $m_h$.

*Proof.* Write $n = m_{h+1}$ and $p = \pi(n)$. Then $d(p) = h$, so $m_h \le p$. If $p > m_h$, truncate the final occurrence of the length-$p$ border at length $m_h$. This makes $m_h$ a proper border at position $n - p + m_h$, strictly between $m_h$ and $n$. Its rank is at least $h + 1$, contradicting firstness of $n$. Hence $p = m_h$. Any earlier return of that prefix would likewise have a proper border of rank $h$ and thus rank at least $h + 1$. Conversely the border at $n$ supplies the return. Overlap is allowed throughout. □

Choose the **maximum rank over all prefixes** of $R$, and its first milestone. P9 gives at most $k - 2$ first-return extensions after the first letter. The remaining suffix makes no further complete return of the last reference: a return, including one at the ultimate endpoint, would have larger rank. The complete word's rank alone is insufficient: rank falls from 3 at `000` to 1 at `0001`.

# 6. Prefix graphs and the exact first-return extension

## 6.1 A finite coloured graph for a concrete reference

The prefix graph $P(B)$ has vertices $0, \dots, m$ and four directed edge labels:

- $\mathsf{F} : j \mapsto \pi(j)$, with $\mathsf{F}(0) = 0$;
- $\mathsf{S} : j \mapsto j+1$ for $j < m$, absent at $m$;
- $\mathsf{K}_a : j \mapsto \kappa_B(j, a)$, for $a \in \{0, 1\}$, where $\kappa$ is the length of the longest prefix of $B$ that is a suffix of $B[: j]a$.

At $j = m$ the matching update uses the full-pattern fallback $\pi(m)$ before consuming the next letter. This is the usual prefix-matching update, here specified by literal prefix/suffix equality. Each vertex has colour

$$(r_j, M_j, \mathrm{next}_j, \mathrm{root}_j, \mathrm{end}_j).$$

The next component is $b_j$ for $j < m$ and END for $j = m$; root and end are Boolean tests. Colours lie in the finite alphabet $Q \times 2^Q \times \{0, 1, \mathrm{END}\} \times \{0, 1\}^2$. A numerical position is **not** part of a colour. No inverse edge is supplied.

## 6.2 Generate one first return

Fix a safe labelled reference $B$. A **safe first-return extension** preserves its word and all labels through position $m$, appends at least one letter, and has its first new complete occurrence of $B$ ending at the new endpoint. All old and fresh positions satisfy the reversed transition, final-root, Bad and border-mask safety conditions of Section 5. Old-label agreement is part of this definition; choosing a new label for an old position is forbidden.

Let $C = Ba_1 \cdots a_L$, $L \geq 1$, be a first-return extension, and label its fresh positions $v_i = m+i$. Define old-graph cursors

$$h_0 = \pi(m), \qquad h_i = \kappa_B(h_{i-1}, a_i), \qquad h_i \neq m \ (i < L), \quad h_L = m. \tag{6.1}$$

Choose reverse-run states $q_0 = r_m, q_1, \dots, q_L$, where

$$\delta(q_i, a_i) = q_{i-1}, \quad q_i \notin \mathrm{Bad}, \quad (q_i, q_i) \notin G, \quad q_i \notin M_{h_i}. \tag{6.2}$$

**Lemma P10 (generator and graph surgery).** Equations (6.1)–(6.2) generate exactly the safe first-return extensions of $B$. Their prefix graphs are obtained by the following finite local rules.

At every old vertex $j < m$, edges and colours stay unchanged. At $m$, change next to $a_1$ and end to false, add $\mathsf{S} : m \to v_1$, and replace its $\mathsf{K}_{a_1}$ edge by $m \to v_1$. Its other $\mathsf{K}$ edge and $\mathsf{F}$ edge stay unchanged.

For $i < L$ the fresh colour is

$$(q_i, \mathrm{col}(q_i) \cup M_{h_i}, a_{i+1}, \mathsf{false}, \mathsf{false}),$$

and the edges are

$$\mathsf{F}(v_i) = h_i, \quad \mathsf{S}(v_i) = \mathsf{K}_{a_{i+1}}(v_i) = v_{i+1}, \quad \mathsf{K}_b(v_i) = \mathsf{K}_b(h_i) \quad (b \neq a_{i+1}). \tag{6.3}$$

At $v_L$, the colour is $(q_L, \mathrm{col}(q_L) \cup M_m, \mathrm{END}, \mathsf{false}, \mathsf{true})$, there is no $\mathsf{S}$ edge, and

$$\mathsf{F}(v_L) = m, \qquad \mathsf{K}_{a_1}(v_L) = v_1, \qquad \mathsf{K}_b(v_L) = \mathsf{K}_b(m) \quad (b \neq a_1). \tag{6.4}$$

*Proof.* Before the first new occurrence of $B$, a proper border of a fresh prefix has length less than $m$; at the first return its longest proper border has length exactly $m$. A longer proper border would contain an earlier return of $B$, as in P9. Hence the proper-border parent at $v_i$ is exactly $h_i$. Equation (5.2) gives its mask and (6.2) checks precisely all new safety conditions. This proves the generator equivalence, including the final return. For matching edges, a matching next letter advances along the fresh strip. A mismatching letter falls back to the proper-border parent $h_i$ and uses its old matching edge. At the new endpoint the proper-border parent is $m$; its matching $a_1$ edge now goes to $v_1$. This yields (6.3)–(6.4). Old proper prefixes cannot refer to new longer prefixes, giving the old-core rules. □

Thus every edge from the old core into the fresh strip enters at **one vertex**, $v_1$. Inside the strip, all edges are forward adjacent, except for the endpoint wrap $v_L \to v_1$. All other fresh edges return to the old core. For $L = 1$, the wrap is a self-loop; it must not be discarded.

### Worked graph example: an overlapping return

Take $B = 0$ and $C = 00$. The occurrence ending at position 2 overlaps the old one, so $L = 1$, $h_0 = 0$, and $h_1 = 1$. There is one fresh vertex 2. Its failure edge goes to 1, its $\mathsf{K}_0$ edge is a self-loop at 2, and its $\mathsf{K}_1$ edge goes to 0. The old endpoint 1 now has both $\mathsf{S}$ and $\mathsf{K}_0$ edges to 2. These statements follow directly by listing the suffixes of `00`, `000`, and `001`.

For a labelled safe example, use three states $f, p, s$ with $\delta(p, 0) = f$, $\delta(s, 0) = p$, terminal final $f$, no Bad states and no flags. The original word `00` runs $s, p, f$. Its reversed labels are $f, p, s$; its two milestones are 1 and 2. The finishing continuation after the single promotion is empty. This realizes the $k - 2 = 1$ boundary exactly.

## 7. Positive directed queries and a finite preimage construction

This section is the central elimination argument. It must operate on a whole conjunction at once: choosing different generated words for different atoms would prove a weaker and unusable statement.

### 7.1 The query language

A path observer has a finite control set $S$, sets of initial and final controls, a Boolean predicate $\mathrm{allow}(s, c)$ on control and colour, and a finite transition relation

$$(s, c, \ell, d, t), \qquad \ell \in \{\mathsf{F}, \mathsf{S}, \mathsf{K}_0, \mathsf{K}_1\}.$$

A graph path is accepted when its vertex/control pairs satisfy allow, consecutive pairs follow an edge and an observer transition, and its endpoint controls are initial/final. Empty paths are allowed

when one control meets both endpoint conditions and allows the vertex colour. No hidden word or vertex register is part of an observer; all its arguments range over finite sets.

A clause existentially quantifies finitely many vertices and conjoins finite unary colour tests, vertex equalities/disequalities, and observer path atoms $\mathrm{Path}_O(x, y)$. A query is a finite union of clauses. Different path atoms have independent existential path witnesses, except that they share the vertices named by clause variables. The language contains neither inverse edges, negated reachability, equality of independent path labels, nor set quantification.

On a finite graph, satisfaction is decidable: enumerate vertex assignments; check the finite tests and equalities; and check each path atom by reachability in the product of the graph with its observer. At most the product's number of vertices is needed for reachability saturation. A query therefore gives a concrete finite decision problem even when its observers are very large.

**Theorem P11 (semantic promotion preimage).** From a finite instance $I$ and query $\varphi$, effectively construct a finite query $\mathrm{Pre}_I\,\varphi$ in the same language, such that for every safe labelled reference $B$,

$$P(B) \models \mathrm{Pre}_I\,\varphi \quad\Longleftrightarrow\quad \exists C\,[\ C \text{ is a safe first-return extension of } B\ \wedge\ P(C) \models \varphi\ ]. \tag{7.1}$$

The compiler takes **only** $I$ and $\varphi$. It does not take $B$, its length, a stack-height bound, or a witness oracle. Terminality is not needed for this local theorem; it supplies the global rank bound later.

The proof has two separate directions: extract a diagram from an actual satisfying extension (coverage, Section 7.7), and reconstruct one extension from a satisfied compiled clause (Section 7.8). Both use the concrete finite envelope below. This is a semantic closure theorem. It does not assert the additional numerical clause-weight estimate in the historical Lean predicate `ClosureFor`. The source map identifies the coverage construction and `compile_sound` separately; the labelled safety part of P10 is also used there.

## 7.2 Why finitely many diagrams cover every satisfying extension

Fix a clause with $v$ variables and $a$ path atoms whose observer sizes are $s_1, \dots, s_a$. Put $\sigma = \sum_i s_i$. For each satisfied atom in $P(C)$, choose an accepting path with no repeated vertex/control pair. Delete any cycle between repetitions to obtain such a path. This preserves the atom's endpoints and tests; its interior is not separately quantified by the clause.

For this product-simple representative, the unique fresh entrance $v_1$ has at most $s_i$ visits. This is not a bound on arbitrary accepting paths. Every fresh excursion after the first must pass through $v_1$, either by an entrance from the core or by a wrap. The path may also start inside the strip. It therefore has at most $s_i + 1$ fresh forward blocks and at most $s_i + 2$ old-core blocks. The total block count over all atoms is at most

$$b = 3a + 2\sigma. \tag{7.2}$$

Mark the first and last fresh vertices, all fresh query-variable positions, and both ends of each fresh block. There are at most

$$g = 2 + v + 2a + 2\sigma \tag{7.3}$$

distinct marked positions. Order them by their physical position and merge coincident positions. Each block crossing a global mark records its observer control there. This includes marks introduced by *other* atoms: their positions are global, even when their controls are atom-specific.

A diagram records these mark groups; the old/fresh location of each query variable; the original-state and next-bit guesses at each group; the first emitted bit; the old endpoint colour; and a finite sequence of core, forward, entrance, exit, and wrap pieces for each atom. It also records the observer controls at piece boundaries and at crossed mark seams. Zero-length core or forward pieces are permitted; the edge pieces still denote one edge.

To make the enumeration literally finite, the checked implementation uses this deliberately generous, reference-independent envelope:

$$\begin{aligned} r &= v + 2ab, \\ z &= r + g + 2b + a + \sigma + 1; \\ \#\text{groups} \le g, &\quad \#\text{core variables} \le r, \\ \#\text{pieces in each atom} \le 2b, &\quad \#\text{seam entries} \le bg. \end{aligned} \tag{7.4}$$

Each integer field in a piece or seam is less than $z$. There must be at least one fresh group, since a first return is nonempty. Enumerate every datum in these ranges, every state/next-bit/colour choice, and every old/fresh location assignment. Filter malformed diagrams by finite consistency tests.

Here is why these bounds suffice. The block argument gives (7.2). Splitting at core/fresh boundaries adds at most one connecting portal edge per block, so $2b$ pieces per atom is safe. At most two old endpoints per block per atom, in addition to the $v$ old variable slots, fit within $r$. Equations (7.2)–(7.3) bound crossed seams by $bg$. The scalar bound dominates every endpoint index, piece index, atom index, and observer-control index. This gives a diagram for any satisfying extension. The enumeration also includes redundant diagrams. It does not claim that all emitted diagrams retain the original product-simple normal form, or a small bound on the number of active observer coordinates. That distinction matters for quantitative estimates.

### 7.3 Pull back the old core

Fix a diagram's first bit $a_1$ and old endpoint colour. In $P(C)$ the old-core induced graph is $P(B)$ with the old endpoint recoloured and its outgoing $\mathsf{K}_{a_1}$ edge suppressed. Define the finite colour map

$$\mathrm{core}_{a_1}(c) = \begin{cases} (c.\mathrm{state}, c.\mathrm{mask}, a_1, c.\mathrm{root}, \mathsf{false}) & c.\mathrm{end}, \\ c & \text{otherwise.} \end{cases}$$

For a core piece, use the same observer controls with fixed entering/leaving controls. Apply this map to both colours of each transition and to its unary allow predicates. Reject $\mathsf{K}_{a_1}$ when its source has the base end bit. This finite observer recognizes exactly that piece in the induced old core, however long the piece is. Old variable tests are pulled back by the same map. Old/old equalities and disequalities remain ordinary base-variable constraints.

### 7.4 Encode marks and the portal edges

For every fresh group $t$, introduce **one** old-graph variable $z_t$ denoting its generator origin $h_i$. Guess one original-state label $q_t$ and one next value $n_t$ for that group. They are reused by every atom touching it. Set

$$\mathrm{fresh}(q, n, c) = (q, \mathrm{col}(q) \cup c.\mathrm{mask}, n, \mathsf{false}, [n = \mathrm{END}]). \tag{7.5}$$

Require the final origin to equal the old endpoint, and all other marked origins to avoid it. Distinct physical groups **may have equal origins**; origins are matching cursors, not positions. Their physical separation is ensured by positive-length intervals below, not by $z_t \neq z_{t+1}$.

Portal pieces are local checks:

- An entrance leaves the old endpoint by $\mathsf{S}$ or $\mathsf{K}_{a_1}$ and reaches the first group.
- A wrap leaves the last group by $\mathsf{K}_{a_1}$ and reaches the first.
- A fresh failure exit reaches its origin $z_t$.
- A proper fresh mismatch on $b \neq n_t$ follows the base edge $z_t\ \mathsf{K}_b\ y$. At the final group, replace $b \neq n_t$ by $b \neq a_1$ and use $z_t = m$.

For each portal, check the original observer transition and endpoint allow predicates using the colour maps. A failure exit becomes equality with an origin and a unary test. A mismatch becomes a single-edge observer over the base graph. Entrance/wrap tests can use the guessed old-endpoint colour and the final origin, so they use only finite unary or edge predicates. No new relation between arbitrary words is introduced. Functionality of base $\mathsf{K}$ edges ensures that two uses of one physical mismatch edge have the same target.

Fresh/fresh equalities and disequalities are decided by group identity. Old/fresh equality is impossible and old/fresh disequality automatic. Pull back fresh variable colour tests through (7.5).

### 7.5 One common generator on every physical interval

Let the distinct fresh groups be $t = 0, \dots, h-1$, from the first fresh vertex to the last. Use two base variables $e, f$ and the one-edge constraints

$$e\ \mathsf{F}\ f, qquad f\ \mathsf{K}_{a_1}\ z_0.$$

Require $e$ to have the guessed old-endpoint colour. The diagram verifies the first reverse transition from that endpoint state to $q_0$ and its Bad/diagonal conditions; the second edge also checks $q_0$ against the origin's mask. Its target is End exactly when $h = 1$. Thus one fresh group handles the single-emission case directly.

For two consecutive distinct groups, all forward pieces traversing that physical interval are known. Split every such piece at all intermediate marks and retain its control at each seam. If one original path crosses an interval twice, it contributes **two coordinates**, not one.

Make a product observer whose control is

$$(q, n, \eta, t_1, \dots, t_b), \tag{7.6}$$

where $q \in Q$ is the current generated reverse state, $n$ is the next bit or END, $\eta \in \{0, 1\}$ records whether a step has occurred, and each $t_j$ is the current control of one active forward piece. Its size is $6k \prod_j |S_j|$. Even when there are no active pieces, retain the generator coordinates; that unobserved interval still has to exist and be safe.

At a base origin of colour $c$, allow a control only if

$$q \notin \text{Bad}, \quad (q, q) \notin G, \quad q \notin c.\text{mask}, \quad (n = \text{END}) \iff c.\text{end},$$

and every active observer allows the single produced colour $\text{fresh}(q, n, c)$. A step can read only a base $\mathsf{K}_a$ edge $c \to d$, with $n = a$ and $c$ not End. Choose $q'$ with $\delta(q', a) = q$, check its safety and the mask of $d$, and choose the next value $n'$ (END exactly at an End origin). Each active coordinate independently chooses an $\mathsf{S}$ or $\mathsf{K}_a$ transition, but **all coordinates see the same $a$ and the same two produced colours**. Set $\eta' = 1$.

Initial and final tuples are fixed by the shared boundary guesses of the diagram, with initial $\eta = 0$ and final $\eta = 1$. This enforces positive length between distinct groups. END has no outgoing generator step. Therefore an earlier full return cannot be hidden in an interval. The next-bit lookahead is a finite guess verified by the next step, shared at adjacent seams.

One regular-path atom between $z_t$ and $z_{t+1}$ now says that one common generator segment realizes every active observation on this physical interval. Concatenate these atoms through their shared endpoint data. Together with the first emission they specify exactly one first-return generation, rather than one generation per original atom. Core path witnesses remain independent, as the original query semantics allow.

### 7.6 Assemble the finite query

Conjoin the pulled-back tests, equalities, portal constraints, core-path atoms, first-emission constraints, and one product-path atom for each interval. Existentially quantify their base vertices. This is a clause of the original query language. Disjoin over the finite diagrams and over the clauses of $\varphi$ to obtain $\text{Pre}_I \varphi$.

Every loop in this construction is over a finite syntax bound, colour alphabet, state set, or observer product. The compiler never enumerates reference words and never needs to bound $|B|$ or $|C|$.

### 7.7 Proof of coverage: a promoted model gives a base model

Suppose a safe first-return extension $C$ satisfies a clause. Fix its variable assignment and accepting product-simple paths. Section 7.2 provides a diagram inside the enumerated envelope. Interpret its fresh groups at their actual positions in the one strip of $C$, and its origin variables at the actual cursors $h_i$. Use their actual states, next bits, endpoint colour and observer controls for all guesses. The old/fresh partition preserves equality, including coincident roles. P10 gives every portal edge and old-core rewrite exactly.

Split the actual generator at the global marks. In each interval, pair this same generator with each active piece's actual observer run. Their product satisfies (7.6), since these observations already traverse those same physical vertices in $C$. Positivity follows from distinct ordered marks. Hence the compiled base clause is satisfied. The satisfying diagram is enumerated by (7.4); no completeness assumption about a heuristic subset of diagrams is used.

### 7.8 Proof of reconstruction: a base model gives one promoted model

Conversely, suppose one compiled base clause is satisfied. Choose witnessing paths for its interval atoms. Their shared origins, reverse states, next values and seam controls match. Together with the first-emission edge, concatenate them to form **one** generator walk. Positivity puts consecutive groups at distinct ordered positions; the End guards force the last emission to be its first complete return. Equations (6.1)–(6.2) therefore give a definite safe reference $C$, and P10 gives its exact graph.

Place every fresh query variable at its shared physical group in this strip, and each old variable at its base assignment. The colour tests and equality partition are correct by construction. Product coordinates supply forward observer paths on each interval of this same strip. Join them across seams for each forward piece. Join these pieces with the core paths and portal edges prescribed by the diagram. The stored controls agree at every join, so each original observer accepts its reconstructed path. Zero-length pieces use their endpoint allow/control tests; a single-group wrap uses P10's genuine self-loop. Thus the original clause holds in $P(C)$, proving (7.1). □

### A complete two-observer diagram

Take four original states $f, p, q, s$, initial state $s$, final state $f$, no Bad states or flags, and only the transitions

$$\delta(p,0) = f, \qquad \delta(q,1) = p, \qquad \delta(s,0) = q.$$

Use the safe reference $B = 0$ with labels $(f, p)$. Its first-return extension $C = 010$ has reversed labels $(f, p, q, s)$. The fresh positions are $v_1 = 2$ and $v_2 = 3$; their old-graph origins are $h_1 = 0$ and $h_2 = 1$. All masks are empty.

The query has two variables $x, y$, tests that their state colours are $q, s$, and two path atoms from $x$ to $y$. Each observer has controls $\{0, 1\}$, initial control 0, final control 1 and unrestricted colour allowance. The first has the single move $0 \xrightarrow{\mathsf{S}} 1$; the second has the single move $0 \xrightarrow{\mathsf{K}_0} 1$. Both hold at $x = v_1, y = v_2$.

Here is every nontrivial guess of a satisfying diagram. There are two fresh groups, for $v_1, v_2$; $x$ belongs to group 0 and $y$ to group 1. Their state guesses are $(q, s)$, and their next guesses are $(0, \mathrm{END})$. The first emitted bit is 1. The guessed old-end colour is $(p, \emptyset, \mathrm{END}, \mathsf{false}, \mathsf{true})$. Each atom has one forward piece from group 0 to group 1, with observer controls 0 and 1. There are no internal seams, core pieces or portal pieces for these two paths. There are no equality or disequality constraints.

Assign the base variables $e = 1$, $f_0 = 0$, $z_0 = 0$, $z_1 = 1$. The first-emission conditions are $e\ \mathsf{F}\ f_0$ and $f_0\ \mathsf{K}_1\ z_0$, since $\pi(1) = 0$ and $\kappa_B(0, 1) = 0$; the reverse-state check is $\delta(q, 1) = p$. There is one physical interval. Its product observer goes from $(q, 0, 0, 0, 0)$ to $(s, \mathrm{END}, 1, 1, 1)$ over the single base edge $z_0\ \mathsf{K}_0\ z_1$. The last two coordinates belong to the two different observers; the third records that a step occurred. The common emitted bit is 0, $\delta(s, 0) = q$, and both observations permit their matching forward edge. The endpoint is the old end, as required by END.

Reconstruction emits first 1 and then 0, obtaining exactly $C = 010$ and both paths in that one graph. Replacing the second observer's sole label by $\mathsf{K}_1$ would remove this product transition: the first observer and the second cannot choose different emissions for the same physical interval.

The literal graph and product transition of this example are checked by the paper's small example checker, separately from the all-input proof.

### Why sharing is indispensable

Suppose two forward observations cover the same interval, one requiring the first emitted bit to be 0, the other requiring it to be 1. Each may have a separately satisfying generated extension. Their conjunction has none. An independent preimage per atom would incorrectly accept; the single product in (7.6) cannot make both transitions on its common first bit. The same issue arises with incompatible lengths, repeated traversals, and marks contributed by another atom. Sharing only endpoints would not repair it.

### Why direction is indispensable

For $B = 0$ and $C = 0\,1^n\,0$, every internal prefix ending in 1 has failure parent 0. If inverse failure edges were available, the old root would have edges to arbitrarily many fresh vertices. The one-entrance proof would fail. Our compiler introduces no inverse edge. Nor does it introduce equality of independent path labels or negative reachability. P11 asserts closure only for the specified positive directed query language.

## 8. Finish without another return

Starting at $h_0 = \pi(m)$ in $P(B)$ with original control $r_m$, follow only $\mathsf{K}_a$ edges. On such an edge from cursor $h$ to $h'$, choose a predecessor control $p$ with

$$\delta(p, a) = q, \quad p \notin \mathrm{Bad}, \quad (p, p) \notin G, \quad p \notin M_{h'}, \quad h' \neq m.$$

The observer also excludes $m$ at its starting cursor and all later vertices; $\pi(m) < m$ because $B$ is nonempty. Accept when the original control is $s$. The empty walk is allowed when $r_m = s$. This is a finite observer with $k$ controls. Introduce an endpoint variable $e$, a variable $f$ with one $\mathsf{F}$ edge from $e$, and a final cursor variable $x$. Choose the endpoint state by a finite disjunction. The resulting query is $\mathrm{Finish}_I$. More explicitly, its clause for state $q$ conjoins the test that $e$ is the endpoint with state colour $q$, the one-edge atom $e\ \mathsf{F}\ f$, and a continuation-observer atom from $f$ to $x$ starting with control $q$. Thus the observer starts at the proper-border cursor with the old endpoint's original control; those are two different coordinates.

**Lemma P12 (finishing).** For a safe reference $B$, $P(B) \models \mathrm{Finish}_I$ iff some safe extension ending at original state $s$ makes no further complete return of $B$.

*Proof.* Before a complete return, every new proper border is shorter than $B$; otherwise truncating it would give an earlier return. Thus the matching cursor lists the longest proper border, and its mask contains exactly all remaining proper-border columns. The new state's diagonal is checked separately. This is precisely the step condition above. Induction over the observer path gives a safe labelled completion, whose final state is $s$. Conversely any such completion determines its matching-cursor path and the required state choices. Empty completion is included. Reversing the completed reference gives an original witness by Section 5. □

## 9. Decide terminal witnesses by finite semantic promotion

Set

$$\Phi_0 = \mathrm{Finish}_I, \qquad \Phi_{h+1} = \Phi_h \vee \mathrm{Pre}_I\, \Phi_h. \tag{9.1}$$

**Lemma P13 (semantic unwind).** For a safe reference $B$, $P(B) \models \Phi_h$ iff there is a chain of at most $h$ safe first-return extensions from $B$ to a reference satisfying $\mathrm{Finish}_I$.

*Proof.* Induct on $h$. At zero the chain is empty. At a successor, the first disjunct permits the shorter chain; P11 applied to the second supplies its first return and a remaining chain of at most $h$ returns. In the reverse direction, an empty chain satisfies the inherited disjunct; otherwise apply P11 to its first return and the induction hypothesis to the remaining chain. Safety of every intermediate reference is part of the first-return relation. □

**Theorem P14 (terminal decision).** Witness existence is decidable for every finite binary instance with terminal finals.

*Algorithm.* Check the empty word directly. If $k \geq 2$, construct $\Phi_{k-2}$. Enumerate both letters $a$ and all $k^2$ pairs of labels $(f, p)$. Retain the safe one-letter references with labels $f, p$, including $\delta(p, a) = f$ and all boundary checks. Evaluate $\Phi_{k-2}$ on each of their two-vertex prefix graphs. Accept iff one test succeeds. For $k = 1$, no nonempty terminal accepting word is possible: its zero and full borders would require two distinct states by P8. An instance with $k = 0$ has no possible initial state.

The enumerated reference has word $a$ and labels $f, p$ at positions 0 and 1. Its prefix graph exposes exactly those positions. Any extension of its label function beyond position 1 is irrelevant; thus this finite enumeration covers the first-letter graph of every nonempty safe reference.

*Termination.* Equation (9.1) performs exactly $k-2$ finite syntactic operations of P11. The remaining enumeration and finite-graph query tests terminate by Section 7.1. This is a complete finite construction, not an unbounded search waiting to discover a witness or a length bound.

*Soundness.* By P13, a successful test reconstructs a finite safe chain and then, by P12, a completed reference ending at $s$. Its reversal satisfies the original witness predicate. Soundness does not use terminality.

*Completeness.* Reverse any nonempty original witness. Choose its maximum rank over **all** prefixes and its first milestone. P8 bounds that rank by $k-1$. P9 expresses the reference up to that milestone as a chain of at most $k-2$ first returns from its first letter. Its remaining completion has no further return, including at the final cut, so P12 makes the finishing query true there. P13 makes $\Phi_{k-2}$ true on the initial one-letter graph, which the algorithm enumerates. The empty witness was checked separately. □

The proof uses only the semantic equivalence (7.1). A historical Lean predicate named `ClosureFor` additionally asks for a specific numerical clause-weight bound. That stronger bundle is not a hypothesis of P14 or P1. The checked semantic compiler and unwind have separate declarations. Counterexamples to a bound on redundant emitted diagrams do not invalidate (7.1).

# 10. Decide the infinite-continuation branch

Now let $I$ be an absorbing instance with $k$ states, without assuming terminal finals. A witness with a safe cyclic future can be extended to a witness ending at a safe final state $q$ with empty incoming flag column, on a safe nonempty cycle $c$ of length $d \le k$. To see this, first take a positive safe extension to enter the empty-column region; absorption keeps that region closed along the chosen continuation. Repeated states in a finite safe graph supply a nonempty cycle of length at most $k$. P7's safe-extension argument preserves the witness while moving to its base. The prefix can remain arbitrarily long; only the cycle is bounded.

Enumerate all states $q$ and binary words $c$ of lengths 1 through $k$. Retain exactly safe cycles from $q$ to itself with $q$ final and its whole incoming flag column empty. For one retained pair, ask whether a safe word $v$ runs from $s$ to $q$ such that the infinite stream $vc^\omega$ has no equal suffixes at an ordered flagged pair of prefix cuts. Diagonal flags are already excluded by safety; targets strictly after $v$ have empty columns.

## 10.1 The periodic suffix criterion

Write $n = |v|$, and let $r$ be the $d$-periodic stream aligned with $c^\omega$ at cut $n$. Suppose $x = vc^\omega$ and $i < j$. If $x[i:] = x[j:]$, it has period $j - i$ from $i$ onwards. For any $t \ge i$, add a sufficiently large multiple of $d(j-i)$ to move beyond $n$. Periodicity of $x$ along $j-i$ and of $r$ along $d$ gives $x(t) = r(t)$. Hence

$$x[i:] = x[j:] \iff [x \text{ agrees with } r \text{ from } i \text{ onwards}] \ \wedge \ [r[i:] = r[j:]]. \tag{10.1}$$

Equality of the last two rotations is checked on their $d$ letters. Distinct phases may give the same rotation when $c$ is not primitive; comparing phase numbers alone would be incorrect.

## 10.2 A complete finite reverse monitor

Scan a prospective prefix from right to left. A monitor state consists of:

1. the current original control;
2. a phase in $\{0, \dots, d-1\}$;
3. a Boolean saying whether the scanned suffix matches its aligned periodic ray;
4. for each phase, a set of original controls at the already scanned later cuts.

Use the full finite carrier, of size at most $2kd\,2^{kd}$. The seed is at control $q$, phase zero, matching true, with $q$ in bucket zero only. Require $q$ safe. For a reverse edge, choose a bit $a$ and safe predecessor $p$ with $\delta(p, a)$ equal to the current control.

While matching, decrement phase modulo $d$. If $a$ is not the cycle letter at that phase, set matching false and reset the phase and buckets. If it matches, reject when $p$ flags a stored control in any bucket whose periodic rotation equals the new phase's rotation. Otherwise add $p$ to that bucket. Once matching is false, keep it false and keep the canonical reset data. Accept exactly the monitor states whose original control is the instance's initial state $s = I.\text{initial}$.

These are effective transitions on finite data. Undefined or unsafe original edges produce no monitor edge. Reachability from the seed to the accepting set therefore terminates by finite graph search.

**Lemma P15 (monitor equivalence).** Fix one of the retained cycles: its word $c$ is nonempty, it is safe, and its base $q$ is final with empty incoming flag column. An accepting reverse path from its safe seed represents exactly a safe prefix $v$ to $q$ with no flagged equal infinite suffixes among its prefix cuts.

*Proof.* While matching, the buckets contain exactly the controls of the later cuts, grouped by phase. Equation (10.1) identifies the new forbidden pairs as precisely a flag from the new source $p$ to a stored target at an equal rotation. Sets suffice; multiplicities do not change this existence test. All old pairs were checked earlier. At the first mismatch from the right, every earlier suffix contains that mismatch and so cannot equal a later suffix by (10.1). No further buckets are needed, but transition and safety checks remain necessary. A previously rejected pair between later cuts cannot be repaired by prepending a mismatch: its two suffixes do not change. Induction proves soundness. For completeness, use the actual cuts and phases of any safe good prefix as the buckets. Every stored member then has a genuine later cut, so a rejected conflict would contradict the assumed absence of flagged equal suffixes. This builds the reverse path. The empty prefix is the seed case. □

### 10.3 Return from an infinite stream to a finite witness

**Lemma P16 (finite extraction).** A good prefix accepted by this monitor gives an original finite witness with a safe cyclic continuation. Conversely, a witness ending at the chosen cycle base gives a good prefix.

*Proof.* For a good prefix of length $n$, each forbidden pair of old cuts has a mismatch in their infinite suffixes. In fact a mismatch occurs before $n + d$ offsets: after $n$ offsets both suffixes lie in the periodic tail, and $d$ further offsets determine all later equality. Extend the prefix along the safe cycle to length at least $2n + d$. Every old target comparison now tests at least $n + d$ letters and contains its mismatch. Every newer target column is empty by absorption. The whole finite word is safe and final and still has an infinite safe future. Alternatively choose $vc^{n+1}$, which ends at the same cycle base and has sufficient length. Conversely, an original witness already contains a mismatch for each flagged pair; appending an infinite continuation cannot erase it. □

**Theorem P17 (infinite-continuation decision).** On a finite absorbing instance, the cycle enumeration followed by the finite monitor tests decides whether some original witness has a safe cyclic continuation.

*Proof.* The cycle reduction covers every such witness. P15–P16 prove both directions for each enumerated cycle. There are finitely many cycles to test, and each monitor is finite. □

## 11. Compose the complete decision

Normalize the expressions by P2, construct the summary instance $I$ of P5, and compute

$$D(\Gamma, s, t) = \neg(\text{InfiniteWitnessCheck}(I) \ \vee \ \text{TerminalWitnessCheck}(I_{\text{term}})). \tag{11.1}$$

P6 supplies absorption without a caller hypothesis. P7 gives the exact split, P14 and P17 decide its branches, P5 identifies their disjunction with normalized non-entailment, and P2 transports it to the original expressions. This proves P1, including unsatisfiable inputs and arbitrary infinite valuations.

Every stage terminates on finite input: syntax traversal, finite relation and subset enumeration, a finite safe-graph transformation, bounded query promotion, finite graph satisfaction, and bounded cycle enumeration with finite monitors. The word lengths quantified in the semantic proofs are not implicit search cutoffs. Classical reasoning may select witnesses inside erased correctness proofs; the executable definitions enumerate concrete finite data and do not use classical choice as a runtime decision procedure.

The current implementation adds proved early returns for reflexivity, a listed normalized premise, unsatisfiability and a separating root address. Failure of the root test invokes (11.1); it never establishes entailment by itself. These guards change performance, not the theorem's scope.

### A computable bound without a useful numerical estimate

As an ordinary consequence of P14, terminal witness existence has a computable uniform length bound at every state count. For $k \geq 1$ there are

$$k(k+1)^{2k}2^{2k+k^2}$$

labelled partial binary instances: choose the initial state, two partial successors per state, final and Bad bits, and ordered flags. Filter terminal finals and decide every remaining instance with P14. For each positive instance, enumerate finite words by length until its decidable original witness predicate succeeds. This terminates on every positive instance. Take the maximum of the resulting lengths, or zero if there are none. Adding one bounds every source's number of visits. There is no instance with zero states; define the bound there as zero.

This is the repository's Q004 computability corollary. It is not the old C080 formula, a resource bound for evaluating (11.1), or evidence of useful speed. An explicit quantitative analysis must separately bound generated syntax, finite relation work, graph evaluation and the chosen machine representation.

## 12. Validation, provenance, and present limits

The theorem and its semantics are read back at source checkpoint c946d86. The final decision module's source is unchanged from 9f7d57c. The declaration map identifies the definitions and checked lemmas supporting P1–P17. The dependency map distinguishes this route from the larger collection of exploratory imports and earlier conditional claims. Ordinary arguments above are intended to stand on their own; the map is an audit aid, not a replacement for them.

The recorded validation for that checkpoint includes the pinned Lean 4.33.1 build, kernel replay, imported-declaration/axiom audits, runtime checks excluding classical choice, and separate semantic readbacks. Full details, exact source hashes and workflow links remain in the C081 completion record. Those are historical results at the stated revision, not fresh proof runs caused by editing this paper. The separate exposition readback compares the central ordinary argument with those definitions. Its scope and remaining limits are recorded there; it is not human peer review.

Native diagnostics completed 25 of 38 attempts under a 20-second per-process cap; 13 timed out. Those timeouts are **UNKNOWN**, never negative entailment or nonexistence answers. All completed direct raw-backend probes in that corpus returned non-entailment; raw true-answer probes

timed out. The checked algorithm itself contains no resource cap. These observations expose severe implementation costs; they are not a limitation on the quantifiers of P1.

The current theorem does not by itself prove:

- a particular complexity class or the historical numerical C080 bounds;
- decidability for an explicit-top signature, multiple competing constructors, or contravariant arrows;
- equality with finite-tree entailment;
- a verified parser or an efficient countermodel-producing command-line solver;
- publication novelty, methodological independence from all prior work, or human peer review.

Countermodels in regular form, other arities, and extensions require their own transfer arguments. The finite-tree example in Section 1 already rules out one tempting transfer. Such arguments are recorded separately rather than added as implicit conclusions of P1.

## 13. Related work and attribution status

The supplied August 31 memo, preserved in papers/sep-topfree.pdf, provides the starting problem, product-address semantics and separating-path characterization. Our proof builds on that route. The focused C081 review and comparison matrix record exact source statements, signatures, variance, tree domains and citation trails.

Niehren and Priesnitz's 2003 automata and word-equation characterization is the closest established comparison. Their Theorem 1 equates non-structural entailment over one covariant constructor **with bottom and explicit top** to restricted cap universality; Theorems 2 and 4 give the automata translations. Their treatment includes finite, regular and arbitrary trees, and identifies the shared word equation behind the cap condition. This establishes prior art for address languages, cap automata and periodic shared words. It is a characterization of the decision problem, not a general decision algorithm. A translation into those automata alone would therefore not subsume P1 under an earlier decidability theorem.

The 1999 restricted result gives PSPACE-completeness for its product-equality and variable-inequality syntax. Its ambient valuations still contain top-labelled trees, although the restricted constraints omit constants. Identity translation already fails on the constant-free premises $x = x \times x$, $x \leq y$ and query $y \leq x$: top-free models force $x = y = \Omega$, whereas $x = \Omega, y = \top$ is a full-signature countermodel. Here $\Omega$ is the full infinite product tree.

There is nevertheless an exact reduction for the input fragment **without bottom constants**. Name product subexpressions, add a fresh equation $\omega = \omega \times \omega$, and guard each name $z$ by $z \leq \omega$. The equation forces $\omega = \Omega$; the guards exclude every top-labelled address. Source models extend to this target and target models restrict back, with the same query value. The construction has polynomial size and uses exactly the restricted syntax. Theorem 14 therefore supplies a PSPACE upper bound for this fragment over arbitrary trees. The ordinary transfer proof states both model maps and the finite-support encoding, and explains the regular-tree consequence. This covers no bottom literals or denial constraints, and the omega equation makes it unsuitable for finite-tree semantics. It also gives no hardness transfer and no PSPACE bound for the full P1 input or its reference implementation. Thus decidability of this restricted fragment should not be claimed as new here.

Other nearby results also have distinct contracts. The Su–Aiken–Niehren– Priesnitz–Treinen first-

order theory result uses a richer logical language; its undecidability is not an undecidability theorem for this entailment fragment. The Breuvart–Penelle–Rehof 2019 slides announce a restricted result and periodic-selection machinery, but the focused review did not locate an archival complete proof to import. The Schubert 2021 preprint was withdrawn because its Lemma 5.5 was formulated incorrectly; its claimed PSPACE upper bound is not an established predecessor theorem.

The candidate contribution is consequently specific: an effective terminal first-return preimage construction preserving an entire conjunction through one shared generated word, combined with the infinite branch and exact normalization into a checked total decision over the bottom/product arbitrary- tree domain. Neither the use of automata nor periodicity is new in isolation. The distinction between the explicit top leaf and the full infinite product tree is substantive: the latter equals its own product, whereas the former does not. The extension study gives a small failure of the naive identification.

This is a bounded novelty review. It follows the main backward and forward lineage and states its search limits; it does not establish publication priority or prove that no equivalent prior algorithm exists. Remaining comparisons include exact reductions to the restricted prior decidability results and verification of the slide-level announcement's full theorem. Those questions are separate from the correctness and termination proof above.